



SPI / I2C

Design & Simulation

[HARMAN] 세미콘 아카데미 2기 | 이윤지



목차

01 개요



02 Serial 통신



03 SPI



04 I2C



05 IP



06 C code & 동작 영상



07 Trouble shooting



08 고찰





CHAPTER 01



개요

● 프로젝트 목표

I2C / SPI 통신 프로토콜 설계 및 시뮬레이션

C application을 작성하여 통신 모듈 구현

FND에 Up Counter를 출력하는 시스템 구현

● 구현환경



C



Vivado



SystemVerilog



Serial 통신

- UART, SPI, I2C는 데이터를 주고받기 위해 사용되는 대표적인 **직렬 통신 방식**
- 각 프로토콜은 배선 수, 속도, 연결 방식이 다르기 때문에, 사용 목적에 따라 선택
- UART은 단순 통신, SPI는 고속 통신, I2C는 다중 장치 제어에 최적화

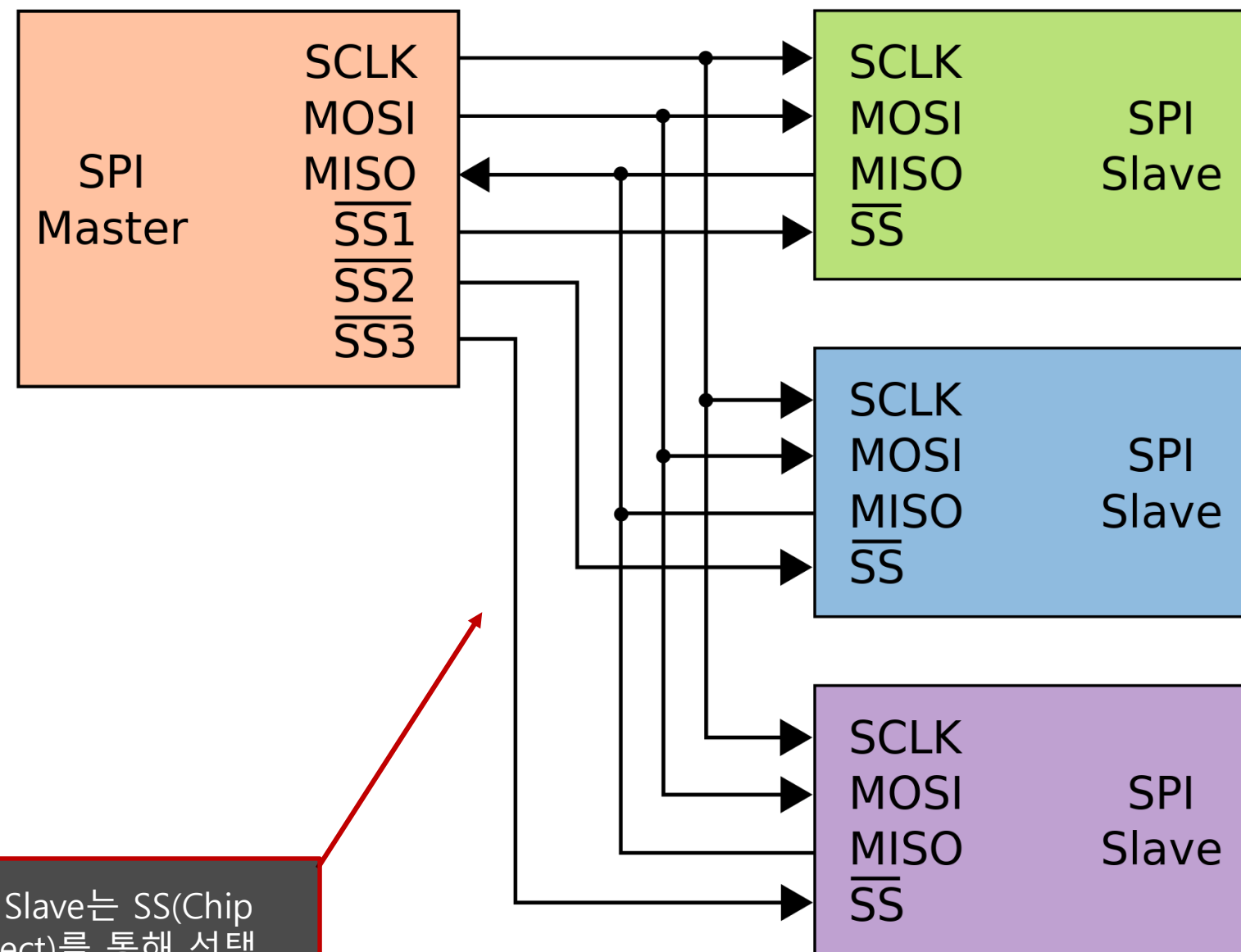
Protocol	UART	SPI	I2C
Type	Asynchronous	Synchronous	Synchronous
전송방식	1:1	1:다수	1:다수
line 수	Tx, Rx	MOSI, MISO, SCLK, CS	SCL, SDA
이중 통신	Full-duplex	Full-duplex	Half-duplex
속도	(Spi보다)느리다	(uart, i2c보다) 빠르다	(spi보다) 느리다
회로 복잡도	Simple	Simple	Complex



CHAPTER 03



SPI



각 Slave는 SS(Chip Select)를 통해 선택

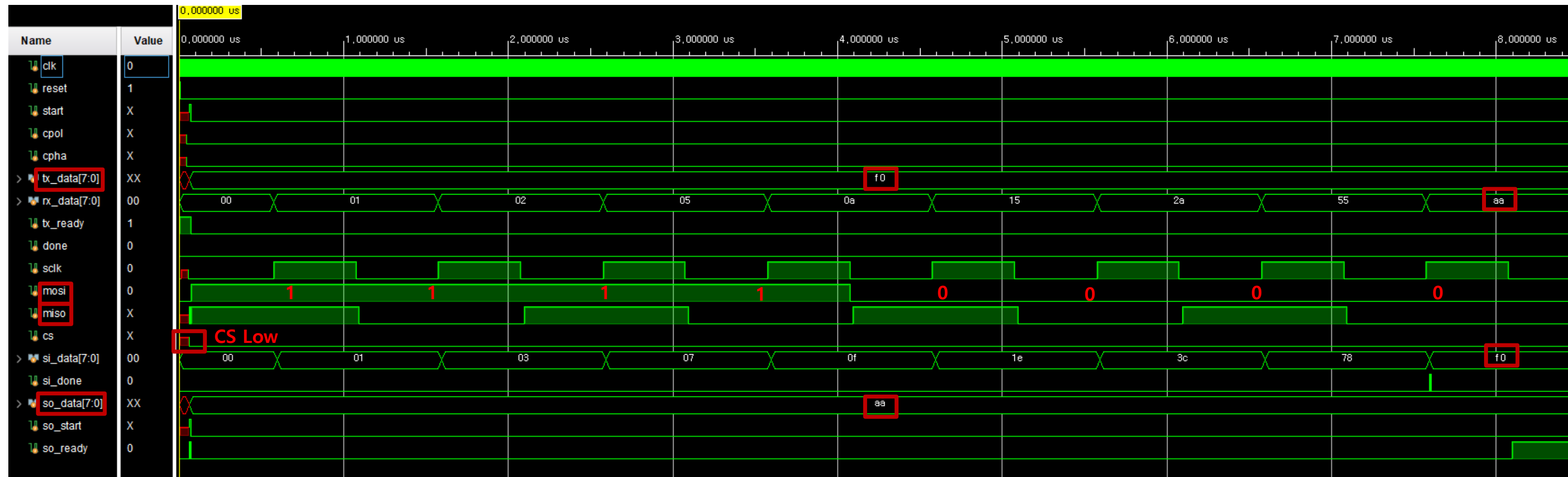
SPI

- Master-slave 방식의 고속 직렬 통신 프로토콜
- 여러 slave 중 하나를 선택하여 양방향 (full-duplex) 데이터 전송 수행
- SCLK, MOSI, MISO, SS(Chip Select) 4개의 신호선
- MOSI : Master -> Slave
- MISO : Slave -> Master



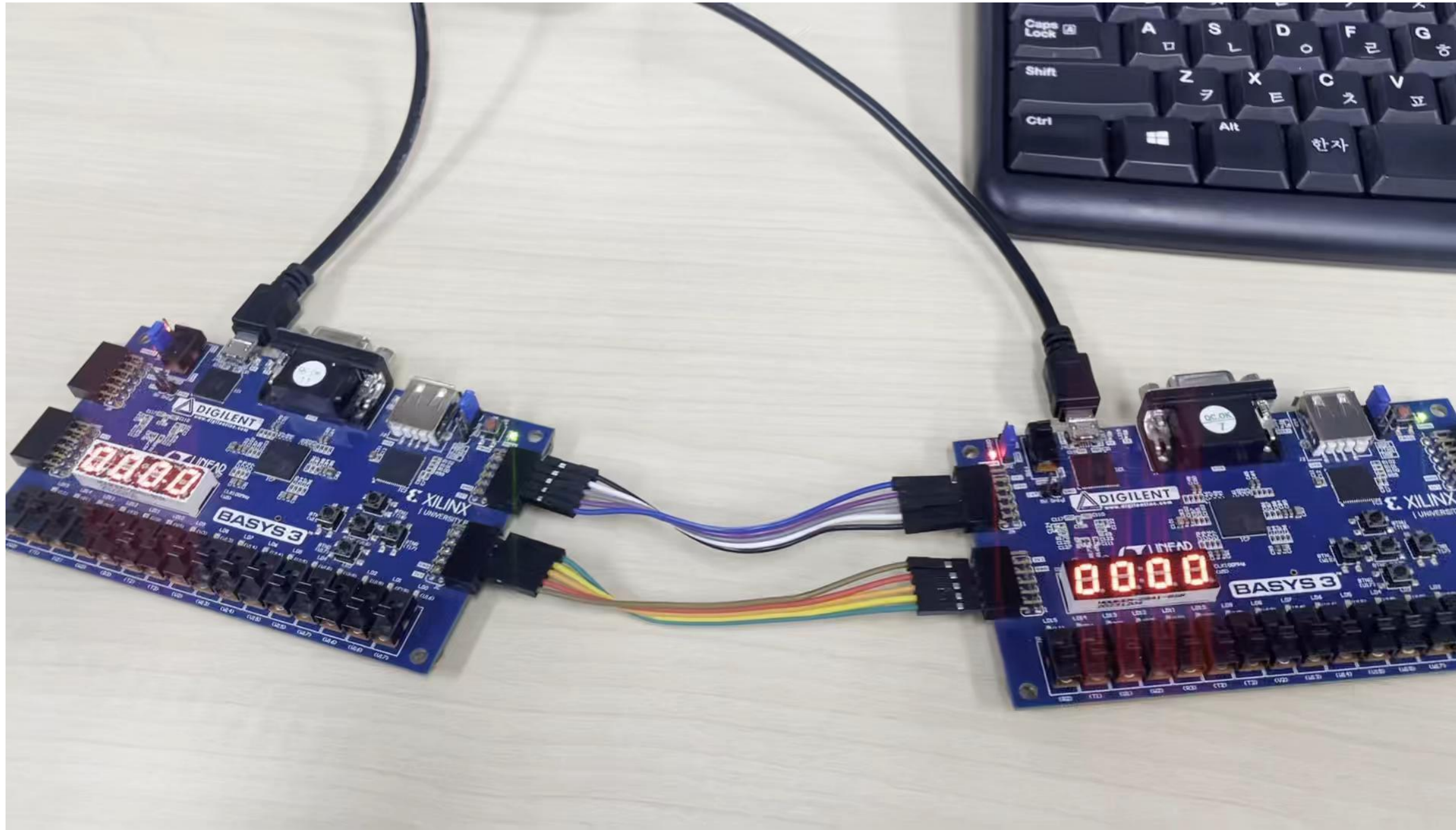
CHAPTER 03

SPI



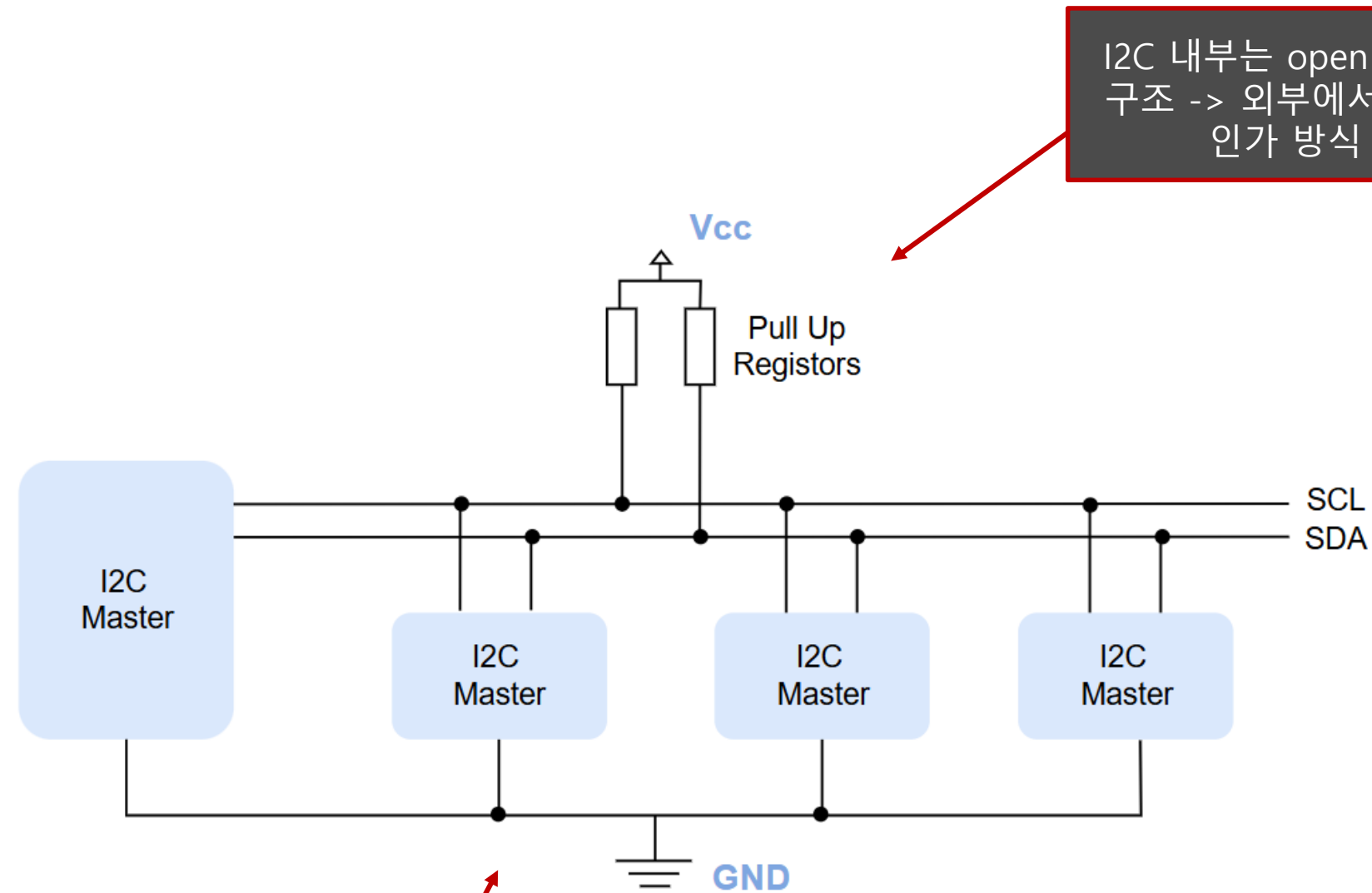
- Cs Low 시점부터 data 통신 시작
- MOSI : Master의 tx_data 로 들어온 f0 이 slave에 전달
- MISO : Slave의 so_data 인 aa를 miso를 통해 master로 전달

SPI



CHAPTER 04

I2C



I2C 내부는 open Drain
구조 -> 외부에서 전압
인가 방식

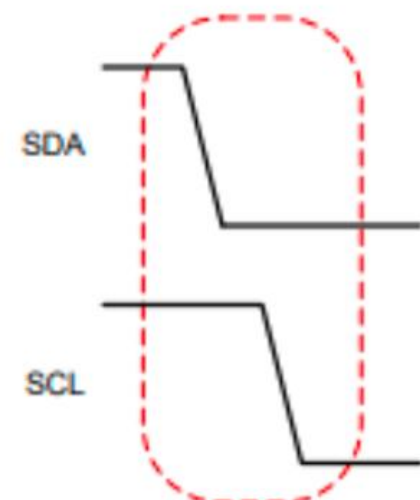
모든 Slave는 고유한
주소값을 가짐

✚ I2C ?

- 두 선으로 여러 Slave와 통신하는 연결 구조
- **SCL, SDA** 두개의 라인으로 통신
- 모든 Slave는 SCL, SDA를 공유
- SCL : Serial Clock
- SDA : Serial Data

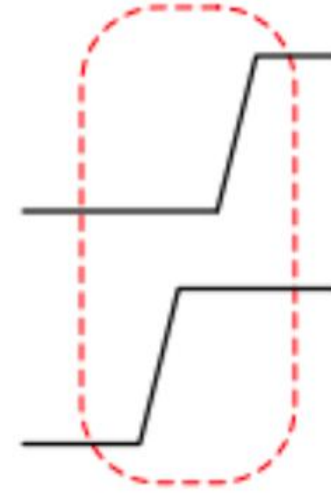
CHAPTER 04

I2C



START

SCL = 1 && SDA Falling



STOP

SCL = 1 && SDA Rising

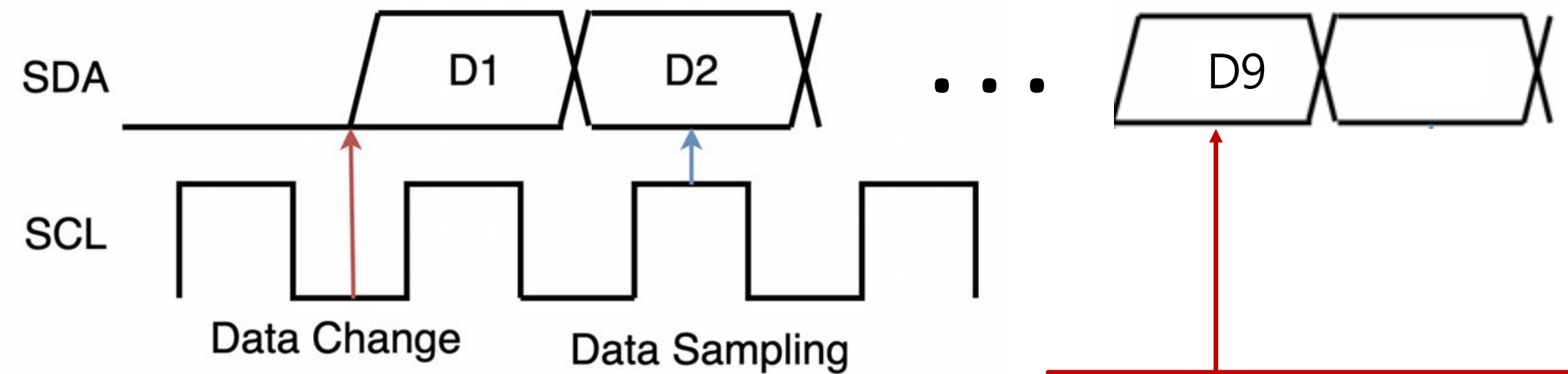
I2C : SCL high 상태에서 SDA가 변할 수 있는 상태 2가지

• STOP / START 조건

공통 조건 : SCL High

Start : SDA High -> Low

Stop : SDA Low -> High



SCL Low : Data Change

SCL High : Data Sampling

I2C : SCL low 상태에서만 data change 가능.

• DATA 전송 조건

SCL Low 일 때만 변경

SCL High에서 샘플링

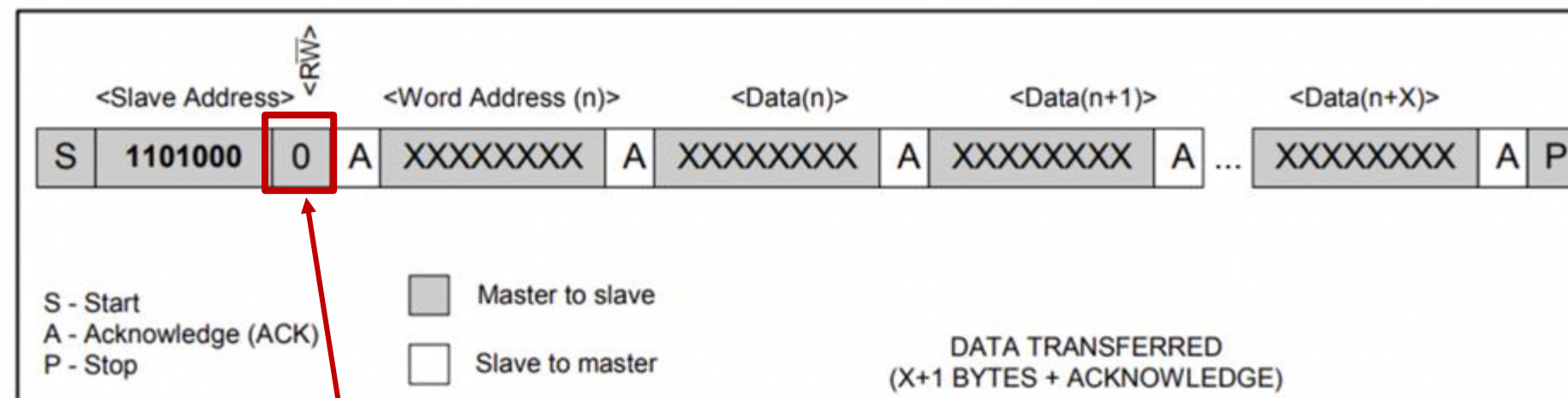
Master가 8bit 보낸 후에
SDA = 0 : ACK
SDA = 1 : NACK 신호 보냄

ACK : 데이터를 성공적으로
받음

NACK : 데이터 수신 오류 /
수신 종료



Figure 4. Data Write—Slave Receiver Mode



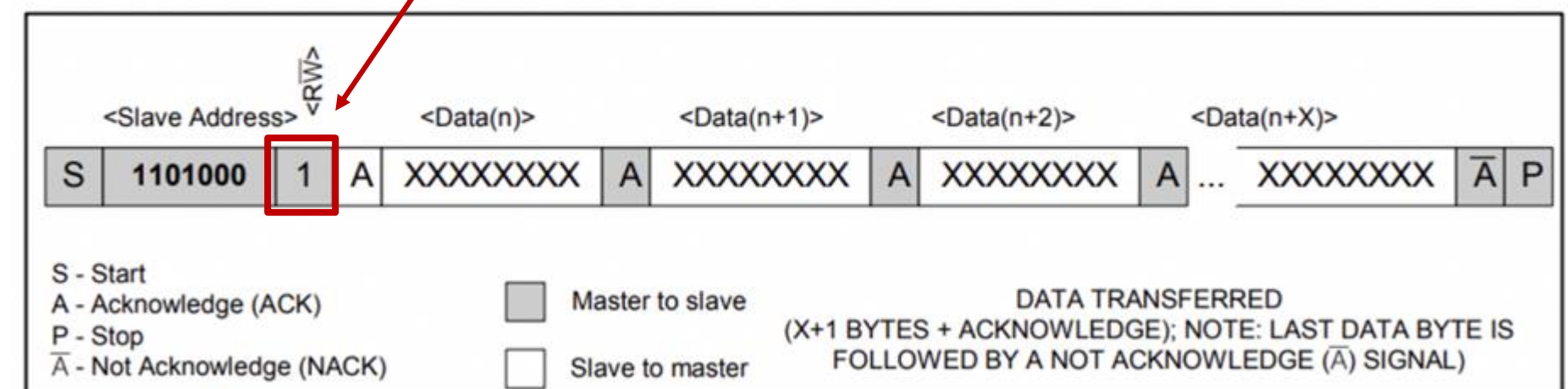
0 : Write

Write

“Master가 데이터를 Slave로 보내고, Slave는 각 바이트마다 ACK로 응답”

1 : Read

Figure 5. Data Read—Slave Transmitter Mode



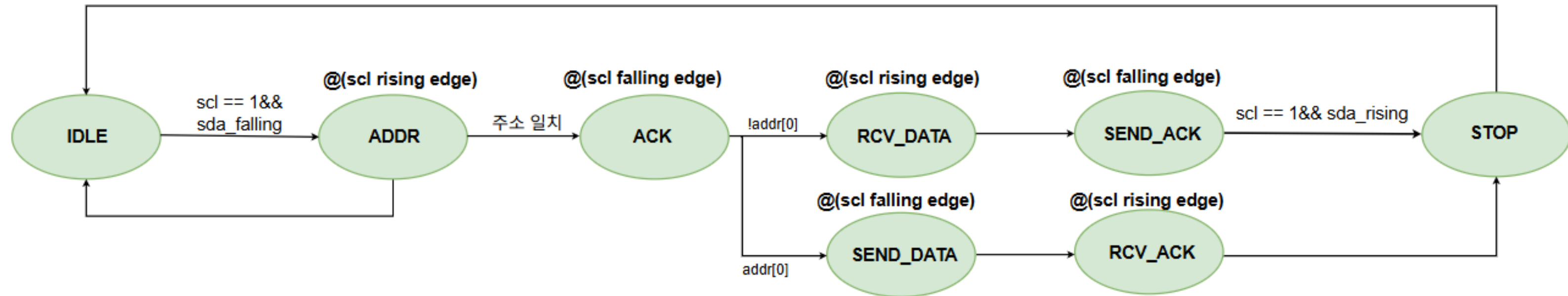
Read

“Slave가 데이터를 Master에게 보내고,
Master는 ACK/NACK으로 수신 여부를 결정”



I2C

[Slave FSM]



Slave는 **Start(SCL=1에서 SDA ↓)** 로 시작해 SCL rising에서 주소/데이터를 읽고, SCL falling에서 ACK나 데이터를 출력
 Stop 조건 (**SCL=1에서 SDA ↑**) 을 감지하면 즉시 IDLE로 복귀

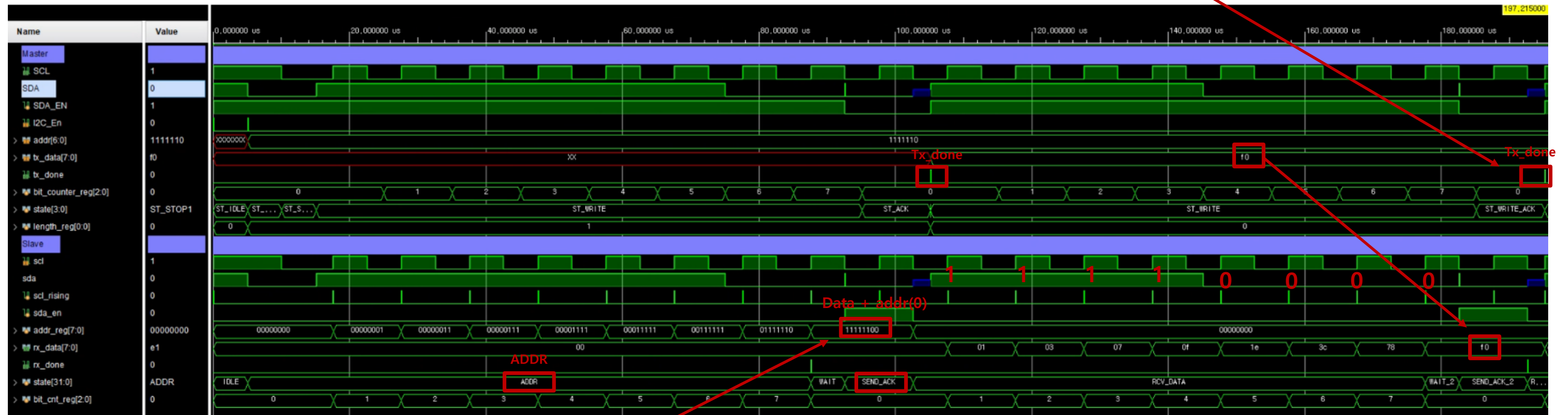


CHAPTER 04

I2C

I2C WRITE Simulation

Byte 단위의 data 전송
완료 후 tx_done



Master → Slave
방향으로 데이터를
1비트씩 전송

12C

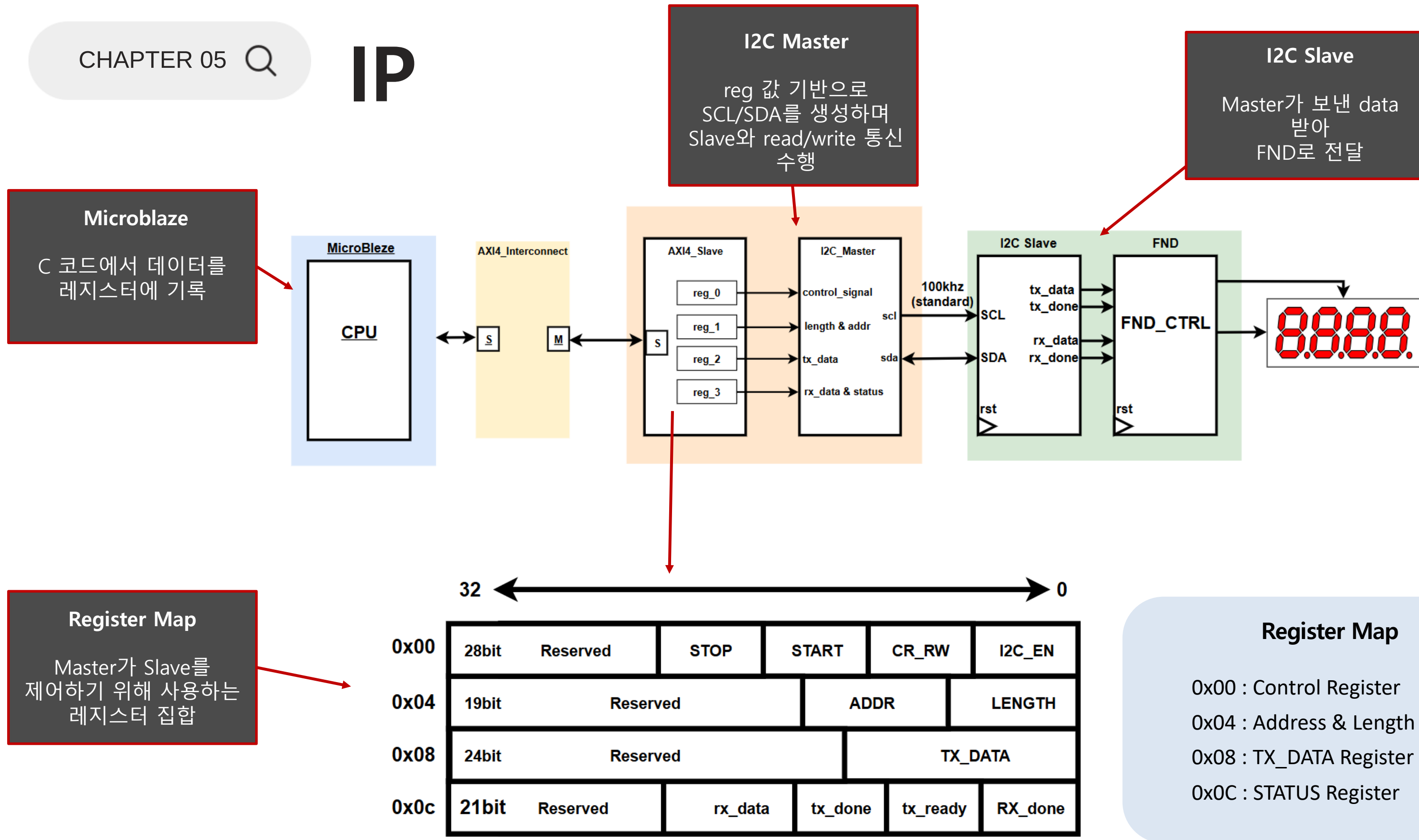
The timing diagram illustrates the I2C communication between a Master and a Slave. The Master's SCL and SDA lines are shown, along with its internal state (ST_IDLE, ST_WRITE, ST_ACK, ST_READ, ST_REA...). The Slave's scl, sda, and o_sda lines are also shown, along with its internal state (ADDR, WAIT, SEND_ACK, SEND_DATA, R..., RCV_ACK_HOLD, IDLE). The diagram shows the Master sending a start condition, address 0x10, and data 0x00. The Slave responds with an acknowledgment. The Master then sends a stop condition. The Slave's internal state is shown as ST_IDLE, ST_WRITE, ST_ACK, ST_READ, and ST_REA....

Slave → Master
방향으로 데이터를
1비트씩 보냄



CHAPTER 05

IP





CHAPTER 06 🔍

C code

```
typedef struct {
    volatile uint32_t CR;           // 0x00
    volatile uint32_t ADDR;        // 0x04
    volatile uint32_t TX_DATA;     // 0x08
    volatile uint32_t RX_DATA_STATUS; // 0x0C
} I2C_TypeDef;
```

I2C IP의 레지스터를 직접 접근
START/STOP, 주소, 데이터 전송을 제어

```
I2C_REG->ADDR =
    ((uint32_t)slave_addr << I2C_ADDR_SHIFT) |
    (length & I2C_LEN_MASK);

I2C_REG->CR = I2C_CR_EN;

xil_printf("ADDR REG = 0x%08lx\r\n", (unsigned long)I2C_REG->ADDR);
xil_printf("CR REG = 0x%08lx\r\n", (unsigned long)I2C_REG->CR);

I2C_WaitTxDone();
```

주소 레지스터 설정 후 EN을 넣어
Master FSM을 시작하고, TX_DONE으로
전송 완료를 확인

```
// value: 0 ~ 9999
static void FND_ShowNumber(uint16_t value, uint8_t sw1)
{
    // sw1 == 1 → STOP
    if (sw1 == 1)
        return;

    // 0~9999
    if (value > 9999)
        value = 0;

    uint8_t buf[2];

    buf[0] = (value >> 8) & 0xFF;
    buf[1] = 0x00;

    I2C_WriteBytes(buf, 2);
}
```

16비트 숫자를 상위/하위 8비트로 나눠
전송, 카운트의 동작 / 정지를 제어
이를 실시간으로 FND에 표시

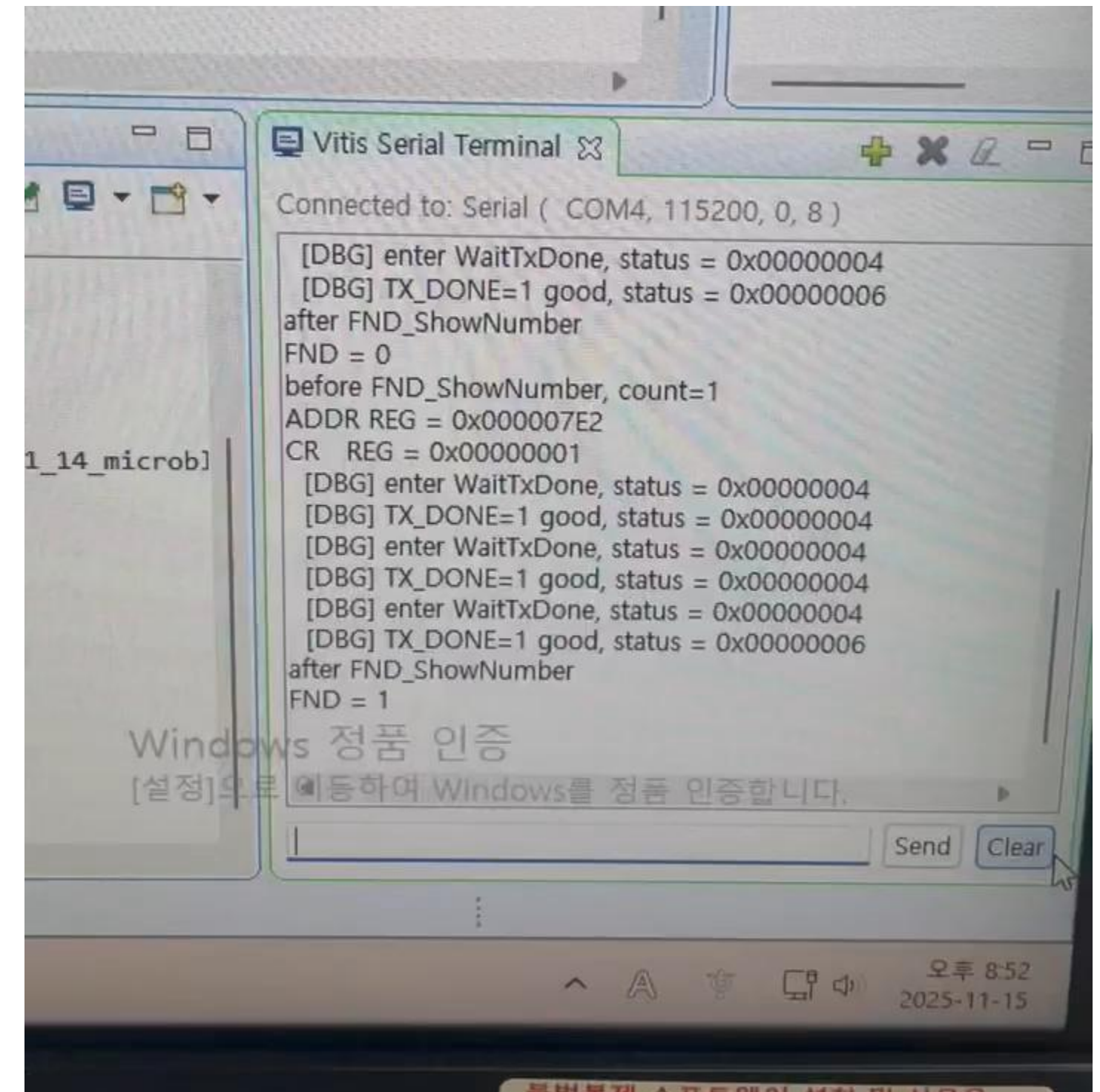
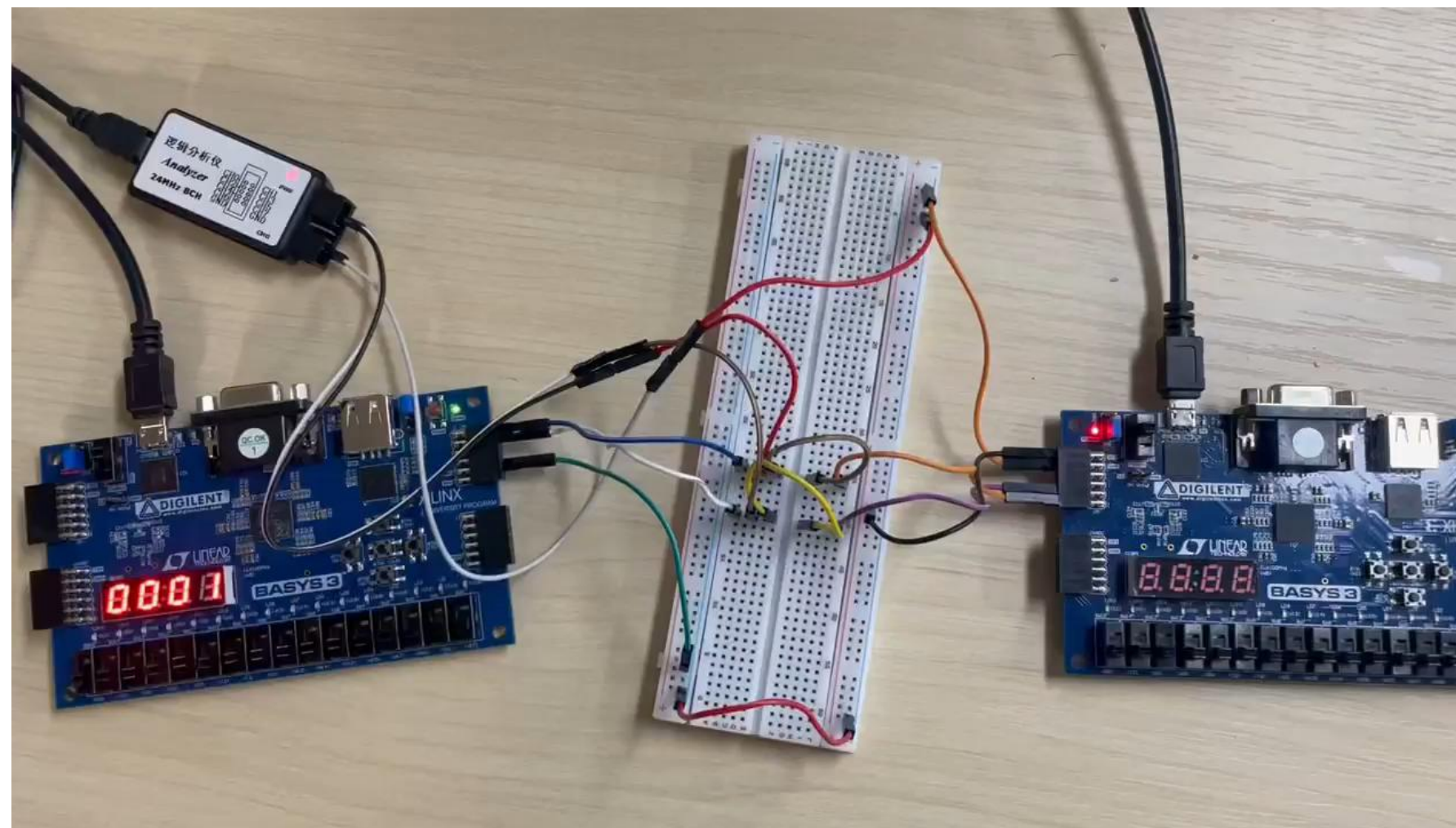
CHAPTER 06 🔍

동작 영상

[콘솔 로그]

[slave 보드]

[master 보드]





Trouble Shooting

문제 발생

- Master와 Slave가 둘 다 ACK 타이밍에 SDA를 구동하려고 함
- SDA 가 'X' 출력 (충돌 발생)
- Slave가 ACK를 내보내야 할 때 sda_en = 1 이지만, 그 타이밍이 Master가 SDA를 사용하는 타이밍과 겹침
- 데이터를 전송한 뒤 ACK 비트를 읽어야 하는데, Slave의 ACK(0)를 SDA에서 관찰할 수 없음.

문제 해결

```
assign sda = (sda_en) ? o_sda : 1'bz;
```

- Slave : **ACK/NACK 구간에서만 sda_en=1**로 SDA를 구동
- Master : **ACK구간에서 SDA_en = 0**로 SDA를 Hi-Z로 변경
- Master를 SDA_en = 0으로 만들어서 Slave가 낸 ACK 가 온전히 반영



고찰

I2C는 START/STOP, ACK/NACK 조건처럼 눈으로 보이지 않는 규칙들이 생각보다 까다롭다는 걸 알게 되었고, SPI는 이러한 규칙에 따라 데이터 타이밍이 달라져서 모드를 맞추는 과정에서 많은 시행착오를 겪었다. 특히, “왜 데이터가 안 가지?”라는 문제를 디버깅하면서 단순한 버그가 아니라 신호의 타이밍이나 state가 바뀌는 시점을 시뮬레이터 파형을 통해 문제를 추적하는 과정이 쉽진 않았지만, 그 안에서 통신 프로토콜이 어떻게 동작하는지 깊게 이해할 수 있었다.



Thank You